

Hashing

Reg Dodds
Department of Computer Science
University of the Western Cape
rdodds@uwc.ac.za

April 25, 2017

Draft

Chapter 1

Hash tables

HASHING is an easily implementable way to construct tables that permit insertion, searching and deletion in *unit time* independent of their size using *direct look up*. Methods for hashing are often built into languages or can be found in readily available libraries. In this chapter we both, on the one hand, illustrate some examples of their use in `python` and `Java` and also, on the other hand, look into the finer details of their actual construction ourselves.

Let us entertain a short diversion to show what is meant by unit time and direct look up and illustrate this with lists and hash tables.

1.1 Lists, and hash tables in `python` and `Java`

Lists and hash tables are convenient data structures found in most languages. We will first discuss lists to fix some ideas about the speed of looking up items and then see how much more efficient hash tables are.

1.1.1 The `list` class in `python`

In `python` the `list` structure is defined as a built-in class with methods such as `append`, `remove`, `lookup`, `pop`, `sort`, and `reverse` that do what their names suggest. Counting from 0, finding the i -th element of L is done by $L[i]$ —noting that the first element is stored at $L[0]$ —this takes *unit time*. The reason why such accesses can be achieved without taking the size of the list into consideration is that in order to look up the i -th item, a base address $baseL_0$ for $L[0]$ must be known, and the offset of the i -th

element is added to this to give the address or position of $L[i]$.

$$\text{address}(L[i]) = \text{base}L_0 + i \times \text{size of element}$$

It is obvious that this calculation is independent of the length of the list. Lists have a great disadvantage otherwise: finding out if an element is in a list with e in L takes *linear time*. In order to discover if e is in the list L every element needs to be inspected sequentially until e is found, or the search is abandoned when reaching the end of the list. In python we can run through all the elements of a list using the `for e in L` construct.

An empty list is defined with

```
1 L = []
```

or with

```
1 L = list()
```

Each of the statements

```
1 K = [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
2 L = [0]*10
3 M = [0 for e in range(10)]
```

produces a list of ten zeros in positions of L numbered from 0 to 9. A list of the first ten numbers starting at 1 is produced by

```
1 N = [r for r in range(1, 11)]
```

Note that 1 is stored at $N[0]$. The sublist of all the odd numbers `odd`, the even numbers `even`, and all the numbers divisible by three can be extracted from this list using splices as follows

```
1 odd = N[0:11:2] # or odd = N[0:10:2] or odd = N[::2]
2 even = N[1:11:2] # or N[1::2]
3 trips = N[2:10:3] # or N[2::3]
```

Two spliced lists can be assigned if they are of equal length. So a splice of a larger list may be set to a splice of a smaller list as long as the splices have the same length. For example, given $P = [0]*10$, the assignment

```
1 P[1::2] = even[:]
```

results in $P = [0, 2, 0, 4, 0, 6, 0, 8, 0, 10]$ and the subsequent splice assignment

```
1 P[::2] = odd[:]
```

results in `P = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`. A splice is turned into a perfect duplicate that is copied over the other splice.

When two lists are assigned this does not happen. Be aware then that the apparently clear assignment

```
1 same = odd
```

does not replace `same` with `odd`, it makes the variable `same` point to the same list as `odd`, such that the assignment

```
1 odd[2] = 111
```

alters the list `odd` to `[2, 4, 111, 8, 10]` but also unexpectedly changes the list `same` to `[2, 4, 111, 8, 10]` because both variables refer to the same list. In order to avert this unexpected behaviour rather assign a duplicate list of the even numbers using the splice `even[:]` to `other` as follows

```
1 other = even[:]
```

This makes `other` point to a copy of `even` and the following assignment

```
1 other[2] = 222
```

now has no effect on the value of the list `even`. Only the list `other` changes to `[2, 4, 222, 8, 10]` and the list `even` remains `[2, 4, 6, 8, 10]`.

The list of the odd numbers less than 100 can be produced with

```
1 odd = [r for r in range(1, 100, 2)]
```

and the reversed list can be produced directly using

```
1 revOdd = [r for r in range(99, 0, -2)]
```

taking care that the final element is 0 and not 1 because the last element of a range is not generated. However, given the list `odd` it can be reversed by

```
1 odd = [r for r in range(1, 100, 2)]  
2 revOdd = odd[::-1]
```

noting that `odd` is not changed.

If we know beforehand where an element lies in a list, it can be looked up by a single constant address calculation that is independent of the length of the list. If, however, the position of an element in a list is unknown elements must be inspected one-by-one until the appropriate one is found. Such an operation depends on the length of the list. If the element is not in the list, all the elements must be inspected before this can be established.

The main points to note are that looking up an element in a list directly can be done in *constant time*—also called *unit time*—only if we know beforehand in which slot it is stored. Otherwise, it takes *linear time* because each element in the list has to be probed before finding it. If the element is not in the list, all the elements in the list need to be probed to discover this.

1.1.2 The dictionary construct in python

In python a dictionary D has methods similar to those of lists, but finding out if x **in** D takes constant time in a dictionary. No matter how large the dictionary becomes, the speed of the look up is fixed. This is similar to knowing beforehand at what position an element e is in a list L . Suppose we are told that the element e lies at position k in the list L , then one can access the element e in constant time by using $e = L[k]$.

An empty dictionary is defined with

```
1 D = {}
```

or with

```
1 D = dict()
```

Consider the statements

```
1 Q = {"the":2, "cat":1, "sat":1, "on":1, "mat":1}
2 R = {123:0, 132:0, 312:0, 321:0, 213:0, 231:0}
3 S = {word for word in "the_cat_sat_on_the_mat".split() }
```

The first two statements produce dictionaries, but the last one produces the set $S = \{'on', 'the', 'sat', 'mat', 'cat'\}$.

In contrast the statement

```
1 L = [word for word in "the_cat_sat_on_the_mat".split()]
```

produces the list $['the', 'cat', 'sat', 'on', 'the', 'mat']$.

Using a dictionary to count the outcomes when shuffling $\{1, 2, 3\}$

In order show the uniformness or bias of a shuffle an interesting experiment is to shuffle the values in the set $\{1, 2, 3\}$ and count the number of times each of the six possible orderings appear. The experiment consists of initializing a list to represent the ordered set as $L = [1, 2, 3]$ and then repeatedly shuffling the list and recording the outcome as a string, such as $r = '123'$ that may be used as the key to update its corresponding entry in the dictionary R , built by the statement

```
1 R = {123:0, 132:0, 312:0, 321:0, 213:0, 231:0}.
```

or by using the following python code

```
1 R = {}
2 R[123] = 0; R[132] = 0; R[213] = 0;
3 R[321] = 0; R[231] = 0; R[312] = 0
```

This code for building **R** initializes each element to zero given its key.

At the end of **N** repetitions

```
1 for repetitions in range(N):
2     r = L.shuffle()
3     R[r] += 1
```

the values of each outcome should be almost the same if the shuffle is unbiased.

Every time that an outcome like '123' occurs it can be counted by **R['123'] += 1**. So it is easy and efficient to count outcomes if we want to establish the bias or uniformness of our shuffling method.

In order to illustrate the power of using a dictionary, we now digress by tackling the problem of counting the outcomes of shuffling the three element list **[1, 2, 3]**, by several attempts to program it without resorting to the use of a dictionary.

Without using a dictionary, a less space efficient arrangement is to store each outcome as a three-digit decimal number and then count the occurrences of these numbers in **L1000** the python **list** with 1000 slots in the range **[0..999]**, and then using the six slots directly. The mapping

$$d_2d_1d_0 \mapsto d_2 \times 10^2 + d_1 \times 10^1 + d_0 \times 10^0$$

will accomplish this. For example, **321** will be stored at **L1000[321]**. Only 6/1000 of the available space has been used.

A slightly more space efficient approach would be to restrict the range to 199 slots by only occupying the slots in the range **[132..321]**. This time we call the list **L199**. The mapping now becomes

$$d_2d_1d_0 \mapsto d_2 \times 10^2 + d_1 \times 10^1 + d_0 \times 10^0 - 123,$$

so that **123** maps to 0, and is stored in **L199[0]**. **132** maps to 9, and is stored in **L[9]**, ..., and **321** maps to 198, and is stored in **L[198]**. This time 6/199 of the space is occupied.

An even more compact method method that uses 27 elements, is to write the three digits as numbers in a base-3 system, where we represent

the outcome $d_2d_1d_0$ as the ternary number, i.e., a base three number that has the digits 0, 1, and 2, with digits $d_2 - 1, d_1 - 1, d_0 - 1$ by applying the mapping

$$d_2d_1d_0 \mapsto (d_2 - 1) \times 3^2 + (d_1 - 1) \times 3^1 + (d_0 - 1) \times 3^0.$$

A slot is reserved in the list for all 27 of the base-3 numbers **111, 112, 113, 121, 122, 123, ..., 333**. This mapping maps **111** onto 0, and **333** onto 26. Slots 0, 1, 2, 3 and 4 are never used and neither of the slots for **322, 323, 331, 332, 333**, i.e., 22, 23, 24, 25, 26, are ever used. By adjusting the mapping slightly to skip the first five elements in order to map **123** onto 0,

$$d_2d_1d_0 \mapsto (d_2 - 1) \times 3^2 + (d_1 - 1) \times 3^1 + (d_0 - 1) \times 3^0 - 5,$$

that is,

$$d_2d_1d_0 \mapsto d_2 \times 3^2 + d_1 \times 3^1 + d_0 \times 3^0 - 18,$$

123 maps to 0, **132** maps to 2, **213** maps to 6, **231** maps to 10, **312** maps to 14 ..., and **321** maps to 16. We have now reduced the necessary storage to hold the numbers to a 17-element list. $\mathbf{U} = [\mathbf{0}] * 17$. Only the six elements 123, 132, 312, 321, 213, and 231 are used; the rest are unused in this application. The portion of the space occupied is now 6/17.

It should be becoming clear that we can do much better by using a *dictionary*, which is also called a *hash table* in python.

Returning to the 27-element implementation $\mathbf{U} = [\mathbf{0}] * 27$, we illustrate the wastage of 21 slots out of 27, noting here that the dictionary implementation only occupies six key-element pairs.

111 $\mapsto U_0$	112 $\mapsto U_1$	113 $\mapsto U_2$	121 $\mapsto U_3$	122 $\mapsto U_4$	123 $\mapsto U_5$
131 $\mapsto U_6$	132 $\mapsto U_7$	133 $\mapsto U_8$	211 $\mapsto U_9$	212 $\mapsto U_{10}$	213 $\mapsto U_{11}$
221 $\mapsto U_{12}$	222 $\mapsto U_{13}$	223 $\mapsto U_{14}$	231 $\mapsto U_{15}$	232 $\mapsto U_{16}$	233 $\mapsto U_{17}$
311 $\mapsto U_{18}$	312 $\mapsto U_{19}$	313 $\mapsto U_{20}$	321 $\mapsto U_{21}$	322 $\mapsto U_{22}$	323 $\mapsto U_{23}$
331 $\mapsto U_{24}$	332 $\mapsto U_{25}$	333 $\mapsto U_{26}$			

The look up needs some extra work. To do the mapping $d_2d_1d_0 \mapsto U_k$, first split the triple into its constituent digits, d_2, d_1 and d_0 and then simply put $k = d_2 \times 9 + d_1 \times 3 + d_0 - 13$, e.g., 312 maps onto U_{19} , since $k = 3 \times 9 + 1 \times 3 + 2 - 13 = 19$.

Python's dictionary does all this work for the programmer and saves *some* of the unused storage.

Counting the unique words in a book

Suppose we want to build a list of all the words in a text and count their occurrences. It is easy and quite efficient to accomplish this with a dictionary. First the whole text must be read from a file.

We have stored the Gutenberg version of James Joyce's *Ulysses* in the file `ulysses.text`. All the punctuation and digits can be removed and a list of `words` can be built by applying the regular expression function `re.findall` on the file.

```
1 from re import findall
2 text = open("ulysses.text", 'r').read().lower()
3 words = findall("[a-z']+", text)
```

This turns all the capitals into lower case letters and extracts the words as lower case strings that may also contain an apostrophe. Next, the dictionary `D` is created. `D` is going to hold the words and their frequencies.

```
1 D = {}
```

Next, the number of appearances of each word in the text is counted.

```
1 for word in words:
2     if word in D:
3         D[word] += 1
4     else:
5         D[word] = 1
```

The unique words can be counted using the code below.

```
1 count = 0
2 for word in D:
3     if D[word] == 1:
4         count += 1
5 print ("Ulysses_has_%d_unique_words" % count)
```

Finally, the word which appears most often, and its number of appearances, can be found by

```
1 mostcommon = "a"
2 for word in D:
3     if D[word] > D[mostcommon]:
4         mostcommon = word
5 print ("The_most_common_word_'%s'_appears_%d_times."
6       % (mostcommon, D[mostcommon]))
```

Finding anagrams in a list of words

Another interesting application of a `python` dictionary is use it to find anagrams. To determine if any two different words are anagrams of one another they must somehow be mapped to the same key in a dictionary. We can call this key the *signature* of the word. This signature can be formed by sorting all the letters of each word into alphabetic order and using this sorted word as the key when storing a word. Every anagram will then map onto the same key. The code must simply check if the word with the given signature is not already in the list and then insert the word. Of course the dictionary entry for the signature may be empty, then a new list of words must be started for that word. The python code follows for finding all the anagrams in Ulysses.

```
1 # This could start off as follows
2 from re import findall
3 text = open("ulysses.text", 'r').read().lower()
4 words = findall("[a-z'+", text)
5 # Ignore hyphenated words---fix
6 anagrams = {}
7 for word in words:
8     s = signature(word)
9     if s in anagrams:
10         anagrams[s] += anagrams[s].append(word)
11     else:
12         anagrams[s] = [word]
13 ...
```

After running this part of the code all the entries in **anagrams** that have 2 words will be anagrams of one another. The code will run in $O(n)$ time since each word is processed exactly once to enter it into the list of anagrams. To determine if a specific entry is an anagram is costs a single lookup in the list of words in **anagrams**.

1.2 Java's classes for hashing

Java has been providing an `int hashCode` function since 1994, see A. Later versions of Java have a `hashCode` class and a `HashMap` class, which provide manu useful methods. We will describe these two classes.

1.2.1 Hashtable versus HashMap

We discuss with some examples the differences between `HashMap` and `Hashtable` objects.

1.2.2 Only Hashtable is Synchronization or Thread Safe

`HashMap` is not synchronized and neither is it thread safe, whereas the class `Hashtable` is thread safe and synchronized. So it is only safe to use `HashMap` when the application never uses any multi-threading tasks, i.e., `HashMap` when using non-threading applications. `Hashtable` should be used when the application requires multi-threading.

1.2.3 Null keys and null values

`HashMap` allows a null key and keys can be associated with null values, whereas a `Hashtable` object may not have null keys or null values.

1.2.4 Iterating the values

An *iterator* is used to process all the values of the `HashMap` object.

```
1 HashMap <Integer, String> m =  
2     new HashMap <Integer, String> ();  
3     m.put(1, "one"); // put a key, value pair into m  
4     m.put(2, "two");  
5     . . .  
6     m.put(10, "ten");  
7     System.out.println("Iterate over HashMap list");  
8     for (Integer key : m.keySet())  
9         System.out.println(key + " : " + m.get(key));
```

For processing a `Hashtable` object an *enumerator* is used, e.g.

<pre> 1 Hashtable t = new Hashtable(); 2 t.put(1, "one"); // put a key, value pair into t 3 t.put(2, "two"); 4 ... 5 t.put(10, "ten"); 6 System.out.println("Display_elementlist"); 7 Enumeration elementlist = t.elements(); 8 while (elementlist.hasMoreElements()) // Display values 9 System.out.println(elementlist.nextElement()); 10 System.out.println("Iterate_over_HashMap"); 11 Enumeration keylist = t.keys(); 12 while (keylist.hasMoreElements()) // Display keys 13 System.out.println(keylist.nextElement()); </pre>	<pre> 1 Display elementlist 2 three 3 six 4 ten 5 seven 6 nine 7 one 8 five 9 four 10 eight 11 two 12 Iterate over HashMap 13 two : 2 14 ten : 10 15 seven : 7 16 five : 5 17 nine : 9 18 one : 1 19 three : 3 20 four : 4 21 eight : 8 22 six : 6 </pre>
--	---

The iterator used by `HashMap` seems to have no obvious advantage over the enumeration used by `Hashtable`. Notice that the elements do not come out in sorted order.

1.2.5 Fail-fast iterator

The iterator in `HashMap` is a fail-fast iterator which means that it is robust—it is resilient to some Exceptions—while the enumerator for `Hashtable` is not. What this implies is that if the `Hashtable` is *structurally* modified after the iterator is created in any way except by using the iterator's own `remove` method, the alterations will cause the iterator to throw a `ConcurrentModificationException`. *Structural* modification means inserting or deleting elements from the `Collection` object, i.e., a `HashMap` or a `Hashtable` object. In this sense the enumerations made from keys or elements in a `Hashtable` are not 'fail fast'.

1.2.6 Performance

Since it is unsynchronized and has to do less computing, `HashMap` uses less memory and is much faster than `Hashtable`. Usually unsynchronized objects run faster than synchronized objects like `Hashtable` in a single threaded environment.

1. The packages needed must be imported.

```
1 import java.util.Collection;
```

```
2 import java.util.Enumeration;
3 import java.util.Hashtable;
4 import java.util.Set;
5
6 public class HashtableDemo {
7
8     public static void main(String args[]) {
9         // The statements below are entered here } }
```

1.2.7 Superclass and Legacy

The Dictionary class has been obsoleted in JDK 1.7 and is not used any more. Hashtable is a subclass of it.¹ The superclass of HashMap is the Map class. Although Hashtable and HashMap have different superclasses they are both implementations of the Map abstract data type.”

1.3 Java's Hashtable class

Java has several constructors for building an initial **Hashtable**. This table behaves almost exactly like python's dictionary. The differences are superficial.

Hashtable()

Constructs a new, empty hashtable with a default initial capacity (11) and load factor (0.75).

The new **Hashtable()** D is instantiated by the statement

```
1 Hashtable D = new Hashtable(<String>, <int>);
```

The keys are strings and the values of the entries are integers.

Hashtable(int initialCapacity)

Constructs a new, empty hashtable with the specified initial capacity and default load factor (0.75).

Hashtable(int initialCapacity, float loadFactor)

Constructs a new, empty hashtable with the specified initial capacity and

¹I don't understand why removing the Dictionary class has any effect on Hashtable because Hashtable still works in JDK 7.

the specified load factor.

Hashtable(Map< extends K, extends V > t)

Constructs a new hashtable with the same mappings as the given Map.

The following points summarize how to use a Hashtable to create a table of capital cities for countries. The table of capitals is used by entering a country and retrieving its capital city.

1. The packages needed must be imported.

```
1 import java.util.Collection;
2 import java.util.Enumeration;
3 import java.util.Hashtable;
4 import java.util.Set;
5
6 public class HashtableDemo {
7
8     public static void main(String args[]) {
9         // The statements below are entered here } }
```

2. Then create a **Hashtable** called **capitals** consisting of capital cities and countries.

```
1 // Create a hash table of capitals
2 Hashtable capitals = new Hashtable();
```

3. Enter a city and its country pair into **capitals**.

```
1 // Enter a country and its capital into Hashtable
2 // capitals.put(key, value) inserts value at key in capitals
3 capitals.put("China", "Beijing");
4 capitals.put("Italy", "Rome");
5 capitals.put("Finland", "Helsinki");
```

Here is complete code example of all above hashtable example or exercises:

```
1 // Hashtable enumeration Example
2 // hashtable.elements() return enumeration of all hashtable values
3 Enumeration enumeration = capitals.elements();
4 while (enumeration.hasMoreElements()) {
5     System.out .println("hashtable_values:_" + enumeration.nextElement()); }

1 // Check if Hashtable is empty in Java
2 // use isEmpty method of hashtable to check emptiness of hashtable in
3 // Java
4 System.out.println("Is_capitals_hashtable_empty:_" + capitals.isEmpty());

—

1 // How to find size of Hashtable in Java
2 // use hashtable.size() method to find size of hashtable in Java
3 System.out.println("Size_of_hashtable_in_Java:_" + capitals.size());

—

1 // How to get all values form hashtable in Java
2 // you can use keySet() method to get a Set of all the keys of hashtable
3 // in Java
4 Set hashtableKeys = capitals.keySet();

—

1 // you can also get enumeration of all keys by using method keys()
2 Enumeration hashtableKeysEnum = capitals.keys();

—

1 // How to get all keys from hashtable in Java
2 // There are two ways to get all values form hashtable first by using
3 // Enumeration and second getting values ad Collection
4 Enumeration hashtableValuesEnum = capitals.elements();
5 Collection hashtableValues = capitals.values();

—

1 // Hashtable clear example
2 // by using clear() we can reuse an existing hashtable,
3 // it clears all mappings.
4 capitals.clear();
```

Output:

Some writers recommend the use of **ConcurrentHashMap** rather than **Hashtable** because it has been extended.

Hashtable was part of the original java.util and is a concrete implementation of a Dictionary.

However, Java 2 re-engineered Hashtable so that it also implements the Map interface. Thus, Hashtable is now integrated into the collections framework. It is similar to HashMap, but is synchronized.

Like HashMap, Hashtable stores key/value pairs in a hash table. When using a Hashtable, you specify an object that is used as a key, and the value that you want linked to that key. The key is then hashed, and the resulting hash code is used as the index at which the value is stored within the table.

The Hashtable defines four constructors. The first version is the default constructor:

```
Hashtable( )
```

The second version creates a hash table that has an initial size specified by size:

```
Hashtable(int size)
```

The third version creates a hash table that has an initial size specified by size and a fill ratio specified by fillRatio.

This ratio must be between 0.0 and 1.0, and it determines how full the hash table can be before it is resized upward.

```
Hashtable(int size, float fillRatio)
```

The fourth version creates a hash table that is initialized with the elements in m.

The capacity of the hash table is set to twice the number of elements in m. The default load factor of 0.75 is used.

```
Hashtable(Map m)
```

Apart from the methods defined by Map interface, Hashtable defines the following methods:

SN	Methods with Description
1	void clear() Resets and empties the hash table.
2	Object clone() Returns a duplicate of the invoking object.
3	boolean contains(Object value) Returns true if some value equal to value exists within the hash table. Returns false if the value isn't found.
4	boolean containsKey(Object key) Returns true if some key equal to key exists within the hash table. Returns false if the key isn't found.
5	boolean containsValue(Object value) Returns true if some value equal to value exists within the hash table. Returns false if the value isn't found.
6	Enumeration elements() Returns an enumeration of the values contained in the hash table.
7	Object get(Object key) Returns the object that contains the value associated with key. If key is not in the hash table, a null object is returned.
8	boolean isEmpty() Returns true if the hash table is empty; returns false if it contains at least one key.
9	Enumeration keys() Returns an enumeration of the keys contained in the hash table.
10	Object put(Object key, Object value) Inserts a key and a value into the hash table. Returns null if key isn't already in the hash table; returns the previous value associated with key if key is already in the hash table.
11	void rehash() Increases the size of the hash table and rehashes all of its keys.
12	Object remove(Object key) Removes key and its value. Returns the value associated with key. If key is not in the hash table, a null object is returned.
13	int size() Returns the number of entries in the hash table.
14	String toString() Returns the string equivalent of a hash table.

Example:

The following program illustrates several of the methods supported by this data structure:

```
1 import java.util.*;
2
3 public class HashtableDemo {
4
5     public static void main(String args[]) {
6         // Create a hash map
7         Hashtable balance = new Hashtable();
8         Enumeration names;
9         String str;
10        double bal;
11
12        balance.put("Zara", new Double(3434.34));
13        balance.put("Mahnaz", new Double(123.22));
14        balance.put("Ayan", new Double(1378.00));
15        balance.put("Daisy", new Double(99.22));
16        balance.put("Qadir", new Double(-19.08));
17
18        // Show all balances in hash table.
19        names = balance.keys();
20        while(names.hasMoreElements()) {
21            str = (String) names.nextElement();
22            System.out.println(str + ": " +
23                balance.get(str));
24        }
25        System.out.println();
26        // Deposit 1,000 into Zara's account
27        bal = (Double)balance.get("Zara").doubleValue();
28        balance.put("Zara", new Double(bal+1000));
29        System.out.println("Zara's new balance: " +
30            balance.get("Zara"));
31    }
32 }
```

Chapter 2

Creating an $O(1)$ look-up table

ASSUME that you have a postal cupboard with m pigeonholes and suppose that these boxes are numbered sequentially from 1 to m . If there N people to share these m postboxes and if $N \leq m$ it is easy to give a different postbox to each person. Simply allocate the postbox numbered i to the person whose $id \equiv i$. When a letter arrives for the person named id , place the letter into postbox $i = id$. This is clearly an $O(1)$ process—it takes a single step to read the id , determine the number of the post box and to place the post into the right box. When person with $ID = id$ comes to get his post, his effort to retrieve the post is also $O(1)$ and he can retrieve his post in $O(1)$ time.

So, on condition that $N \leq m$, it is possible to assign a unique identifier, $id \in [1..m]$, to each of the N persons to yield an $O(1)$ retrieval method. As long as the the identifiers $id \in [1..m]$ occupy the set $[1..m]$ densely this method may be useful. Many identification numbers use a final digit called a check digit that is calculated according to some formula that makes it easy to detect when digits are transposed. So when using such identification numbers the last digit must be omitted when accessing the array $d[]$ —using it will cause the table to be 90% empty.

Sometimes the situation is more complex and there is no obvious means of mapping the set $I = \{id | id \in [1..N]\}$ sensibly to the set $[1..m]$.

We then resort to using a mapping h that is know as a hashing function to do the mapping

$$h : I \rightarrow [1..m].$$

The most important property of this function is that its values must be uniformly distributed.¹

¹Uniformly distributed means that the probability $P(h(i) = x) = k$ for any $i \in I$ and

On the other hand when $N > m$ such an allocation is impossible and there will be unavoidable clashes or collisions. Before completing the discussion of the $N \leq m$ case let us first consider how to handle collisions when N exceeds m . It is inevitable that some slots must be shared. The spill is handled by storing the excess data in chains of keys that map to the same slot. The mapping is quite simple. The key must somehow be transformed into an integer called *number*. The mapping, which we call *h* must map *number* into $[1..m]$. This is accomplished by

$$i = \text{number} \% m + 1.$$

It is very likely that different keys, even though they produce different numbers will in fact sometimes yield duplicate *i* values.

We call the ratio of entries to slots, i.e., N/m , the *load factor*, and designate it by α . The load factor gives the “fullness” of a hash table. Obviously when $N > m$ the load factor exceeds 1. A consequence of having $\alpha > 1$ is that some of the slots will have to hold more than one key.

2.1 $O(1)$ Searching, Insertion and Deletion

- Assume we assign a unique identifier, $id \in [1..m]$ to each of the n persons in the class—in the order in which persons arrive in the class, or any other order.
- If the data for person with identifier *id* is stored at $d[id]$, then any person’s data may be retrieved given their *id* in $O(1)$ time.
- Inserting a new arrival’s data entails first assigning the next available *id* and by inserting his data into $d[id]$.
- Insertion and deletion can obviously be done in $O(1)$ time, and $n = m$.
- Deletion is similarly efficient. To delete an item: (1) it must be marked as deleted, and (2) its *id* must be added to the list of available *ids*, so that this *id* is later available for re-use.
- The method is not very practical if the identifiers have already been allocated—we will *hash* the *ids*.

$k \in I$. This is still nonsensical.

2.2 O(1) Searching, Insertion and Deletion

Suppose there are N persons and we have a table of m positions, such that $N \leq m$. A solution is to devise a function $h(id)$ where $h()$ is called a *scatter* or more often a *hash* function.

A *hash* function where $n == m$ is called *perfect*.

Insertion is done more or less as follows:

```
i = hash(id);
while (d[i].isFilled)    // while might not
    i = (i + p) % m + 1;  // terminate
d[i].key = id;
d[i].isFilled = true;
```

Searching for id is similar:

```
i = hash(id);
while (d[i].isFilled && d[i].key != id)
    i = (i + p) % m + 1;
if (!d[i].isFilled)
    return notFound
else
    return i;
```

2.3 Hash functions:

Let $i_j = \text{hash}(id_j)$ and $i_k = \text{hash}(id_k)$. When $id_j \neq id_k$ but $i_j = i_k$, this is known as a *collision*.

Design the hash function to minimize collisions.

- Attempt to spread the range of $h(id)$ uniformly over $[1..m]$.
- $h(id)$'s values must be as random as possible.

Since collisions are often unavoidable methods of handling collisions must be devised.

Use of linear probing: suppose id_j is already in the table and $i_j = i_k$, then map i_k onto some multiple r of a suitable prime p , i.e. $i_k = (i_j + r \times p) \bmod m + 1$.

The $\bmod m + 1$ at the end ensures that $i_k \in [1..m]$

2.4 The time complexity of *open* hashing

In the literature this method has been given the confusing name of *open* hashing. We will consider *closed* hashing later.

The speed of the table depends on how “full” it is. The fullness of a table is defined in terms of its *load factor*, $\alpha = \frac{N}{m}$. For *open* hash tables $\alpha = \frac{N}{m} \leq 1$, i.e., $N \leq m$. Assume that there are k entries in the hash table, then the load factor is $\alpha = \frac{k}{m}$. When no entries have been made, then the a single probe is enough to make the first entry.

Suppose k entries have been made, then the probability of a collision is α . The probability of a collision on the i -th probe is α^i . The probability of finding an entry in precisely i probes is $\alpha^{i-1}(1 - \alpha)$.

2.5 Search length in *open* hash table

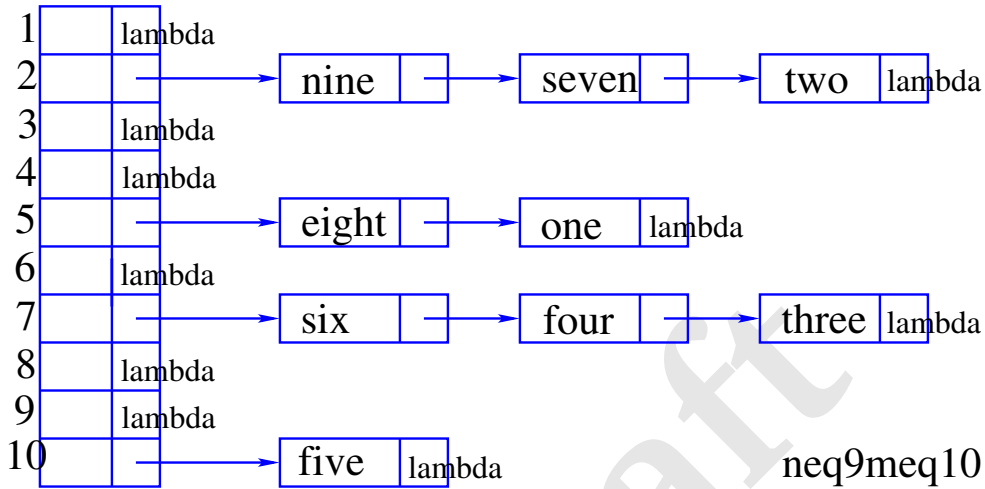
Search length is another name for the average look-up time and is usually measured as the number of probes required to reach an entry in a table. A simple empirical method of determining the average look-up time is to look up each entry in a table populated with N entries once, total the number of probes and divide the total by N to give the average. Suppose k entries have been made, then the probability of a collision is α . The probability of a collision on the i^{th} probe is α^i . The probability of finding an entry in precisely i probes is $\alpha^{i-1}(1 - \alpha)$.

*Average Search Length*_{hash}

$$\begin{aligned}
 &= \sum_{i=0}^N i \alpha^{i-1} (1 - \alpha) \\
 &\leq (1 - \alpha) \sum_{i=0}^{\infty} i \alpha^{i-1} \\
 &= \frac{(1 - \alpha)}{(1 - \alpha)^2} \\
 &= \frac{1}{1 - \alpha}
 \end{aligned}$$

Loading %	α	Average Search Length
85%	0.85	$= 1/0.15 = 6.667$
95%	0.95	$= 1/0.05 = 20$
99%	0.99	$= 1/0.01 = 100$
99.9%	0.99	$= 1/0.001 = 1000$

2.6 Chained hash tables

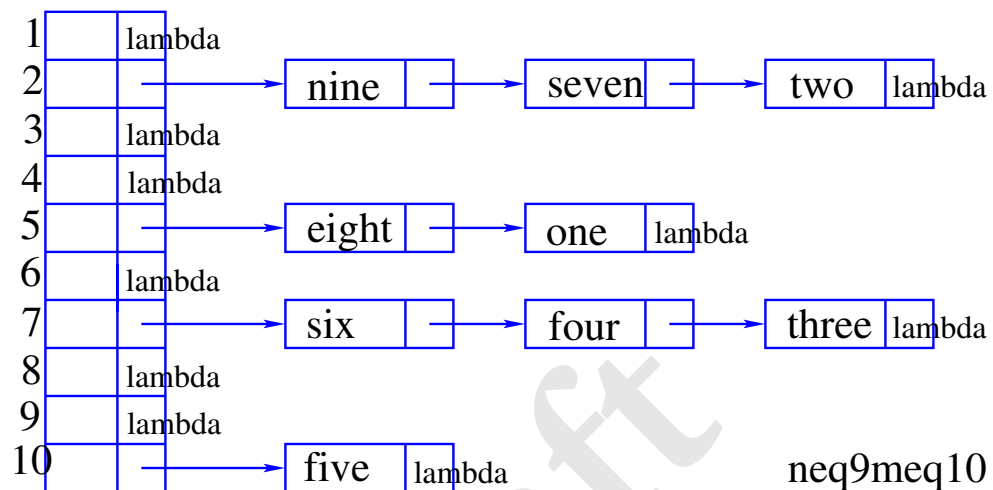


The table $d[m+1]$ has m positions and N entries. In chained tables N may exceed m . The performance of a *chained* hash table depends on the average length of the chains, $1 + \frac{N}{m}$. So the performance may be tuned. One probe is always required, to get to the head of the chain, and then a linear search is done in a list of average length $\frac{N}{m} = \alpha$ is required. This amounts to an average of $1 + \frac{\alpha+1}{2}$ probes to find any item. So $ASL_{\text{chained hash}} = O(\alpha)$

2.7 Chained hash tables can be tuned to $O(1)$

The performance of a *chained* hash table depends on the average length of the chains, $1 + \frac{n}{m}$. So the performance may be tuned. One probe is always required to get to the head of the chain, and then a linear search is done in a list of average length $\frac{n}{m} = \alpha$ is required. This amounts to an average of $1 + \frac{\alpha+1}{2}$ probes to find any item. So $ASL_{\text{chained hash}} = O(\alpha)$ But since α can be made independent of the number of elements n in the list, e.g. putting $6m = N$, makes $\alpha = \frac{N}{m} = 6$, i.e. hashing can be made to be $O(1)$.

2.8 Chained hashing—example

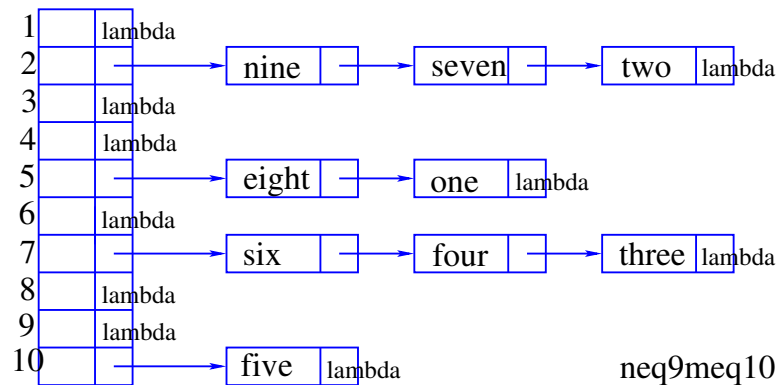


Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order. Other orders will give logically equivalent tables, but their links will be different. Initially there are $N = 0$ items in the store.

Let us enter two, assume $id = \text{"two"}$, so, first hash it: $i = \text{hash}(id);$

For this table $i = 2$. The data, id , may be placed at the next open place, $++n$, in the store, by: $\text{store}[n] = \langle id, 0 \rangle;$ Check if the table entry 2 is open or filled by examining $d[i]$. If $d[i] \neq 0$, i.e. $d[i]$ is already occupied, the link 0 in the node $\langle id, 0 \rangle$ must be replaced by $d[i]$ as in $\langle id, d[i] \rangle;$, and $d[i] == n;$.

2.9 Chained hashing—example: Enter five

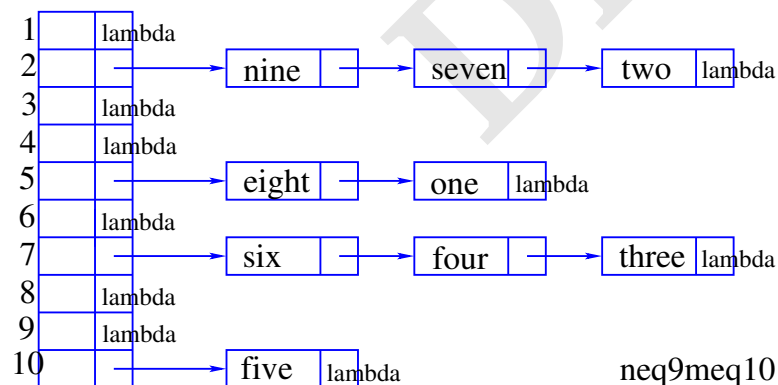


Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

Now enter five.

There is now $n = 1$ item in the store. Entering five, assume $id = \text{"five"}$, so, first hash it: $i = \text{hash}(id)$; For this table $i = 10$. The data, id , may be placed at the next open place, $++n$, making $n = 2$ in the store, by: $\text{store}[n] = \langle id, 0 \rangle$; Check if the table entry i is open or filled by examining $d[i]$. Since $d[i] == 0$, i.e. $d[i]$ is *not* occupied, the link 0 in the node $\langle id, 0 \rangle$ is used as in $\langle id, 0 \rangle$; , and $d[i] = n == 2$;

2.10 Chained hashing—example: Enter three

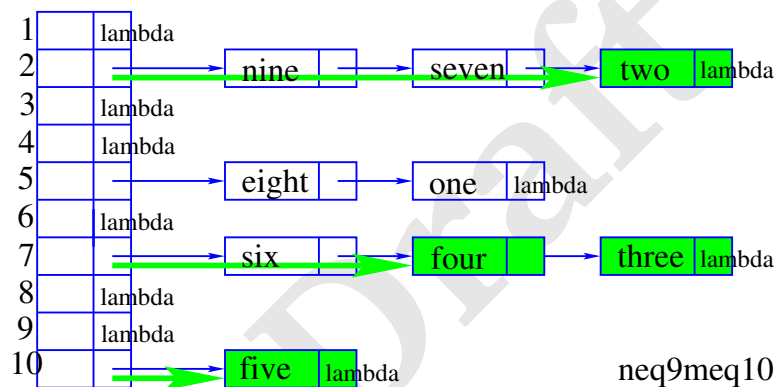


Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

Next enter three.

There are $N = 2$ items in the store. Entering `three`, put `id = "three"`, so, first hash it: `i = hash(id)`; For this table `i = 7`. The data, `id`, may be placed at the next open place, `++n`, making `n = 3` in the store, by: `store[n] = <id, 0>`; The table entry `d[i]` is still open, so the link 0 in the node `<id, 0>` is used and `d[i] = n == 3`.

2.11 Chained hashing—example: Enter four

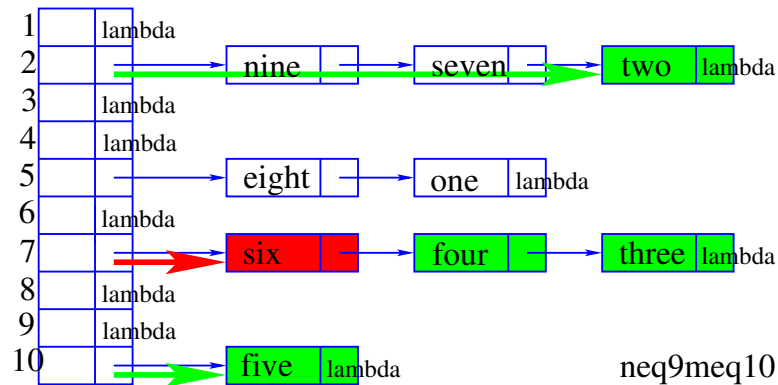


Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

Now enter `four`.

There are now $N = 3$ item in the store. Entering `four`, assume `id = "four"`, so, first hash it: `i = hash(id)`; For this table `i = 7`. The data, `id`, may be placed at the next open place, `++N`, makes `N = 4` in the store, by: `store[N] = <id, 0>`; Check if the table entry `i` is open or filled by examining `d[i]`. Since `d[i] != 0`, i.e. `d[i]` is *occupied*, the link 0 in the node `<id, 0>` must be replaced by `d[i]` as in `<id, d[i]>`; , and `d[i] == N`;

2.12 Chained hashing—example: Enter **six**

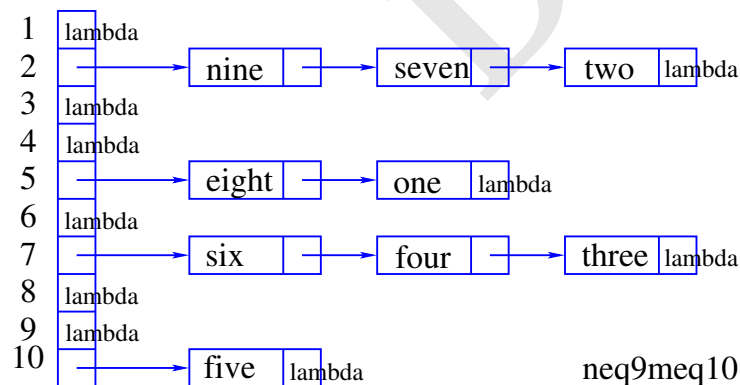


Set up the hash table by entering keys: two, five, three, four, six, one, seven, nine, eight in this order.

Now enter **six**.

There are now $N = 4$ item in the store. Entering **six**, assume $id = \text{"six"}$, so, first hash it: $i = \text{hash}(id)$; For this table $i = 7$. The data, id , may be placed at the next open place, $++N$, makes $N = 5$ in the store, by: $\text{store}[N] = \langle id, 0 \rangle$; Check if the table entry i is open or filled by examining $d[i]$. Since $d[i] \neq 0$, i.e. $d[i]$ is *occupied*, the link 0 in the node $\langle id, 0 \rangle$ must be replaced by $d[i]$ as in $\langle id, d[i] \rangle$; and $d[i] == N$;

2.13 Chained hashing— hashInsert—array implementation



```
int d[] = new int [m+1];
```

```

node store[] = new node [6*m +1];

int n = 0;

void hashInsert(string id) {
    ++N;
    store[N].key = id;
    i = hash(id);
    store[N].link = d[i];
    d[i] = N; }

```

2.14 Chained hashing—hashSearch

This method is called as in the example:

```
boolean found = hashSearch("ten");
```

The method follows

```

boolean hashSearch(string id) {
    int i = hash(id);
    i = d[i];
    while (i != 0 && store[i].key != id) {
        i = store[i].link; }
    return store[i].key == id; }

```

2.15 Chained hashing—hashInsert—list implementation

```

int d[] = new int [m+1];
node store[] = new node [6*m +1];

int n = 0;

void hashInsert(long key, String id) {
    node N = new node (key, id);
    i = hash(id);
    n.next = d[i];
    d[i] = N; }

```

2.16 Chained hashing—hashSearch—using lists

This method is called as in the example:

```
boolean found = hashSearch(10L);
```

The method follows

```
boolean hashSearch(long key, String id) {
    int i = hash(key);
    node here = d[i];
    while (here != null && here.key != key) {
        here = here.next;
    }
    return here != null && here.key == key;
}
```

2.17 Hash functions

2.17.1 Example of HashMap and Hashtable

```
import java.util.Hashtable;
public class HashMapHashtableExample {
    public static void main(String[] args) {
        Hashtable<String,String> table = new Hashtable<String,String>();
        table.put("LV", "fifty five");
        table.put("IC", "ninety nine");
        System.out.println("Hashtable object: " + table);
        HashMap mapobj = new HashMap();
        map.put("Alive is ", "awesome");
        map.put("Love", "yourself");
        System.out.println("HashMap object: " + map);
    }
}
```

Output : Hashtable object output :Love=yourself, Alive is =awesome
 HashMap object output :Alive is =awesome, Love=yourself

Similarities Between HashMap and Hashtable

1. Insertion Order : Both HashMap and Hashtable does not guarantee that the order of the map will remain constant over time. Instead use LinkedHashMap, as the order remains constant over time.
2. Map interface : Both HashMap and Hashtable implements Map interface .
3. Put and get method : Both HashMap and Hashtable provides constant time performance for put and get methods assuming that the objects are distributed uniformly across the bucket.
4. Internal working : Both HashMap and Hashtable works on the Principle of Hashing . We have already discussed how hashmap works in java .

When to use HashMap and Hashtable?

1. Single Threaded Application

HashMap should be preferred over Hashtable for the non-threaded applications. In simple words , use HashMap in unsynchronized or single threaded applications .

2. Multi Threaded Application

We should avoid using Hashtable, as the class is now obsolete in latest Jdk 1.8 . Oracle has provided a better replacement of Hashtable named ConcurrentHashMap. For multithreaded application prefer ConcurrentHashMap instead of Hashtable.

Recap : Difference between HashMap and Hashtable in Java

HashMap Hashtable

Synchronized No Yes

Thread-Safe No Yes

Null Keys and Null values One null key ,Any null values Not permit null keys and values

Iterator type Fail fast iterator Fail safe iterator

Performance Fast Slow in comparison

Superclass and Legacy AbstractMap , No Dictionary , Yes

2.18 Traversing the entries in a HashMap

Traversal of Java dictionary, i.e., `HashMap` entries can be done in several ways.

```
1 Map <String, Integer> D;  
2 @Before  
3 public void setup() {  
4     D = new HashMap <>();  
5     D.put("Beijing", 86);  
6     D.put("Cape_Town", 27);  
7     D.put("London", 44);  
8     D.put("Moscow", 7);  
9     D.put("New_York", 1);  
10    D.put("Paris", 33); }
```

This is a dictionary in Java. Each key is a `String` that is the name of a city. The values are an `Integer` that gives the dialing code for that city.

2.18.1 Traversal over the cities with an Iterator

```
1 @Test  
2 public void iteratorTraversal() throws Exception {  
3     List <String> cities = new ArrayList <>();  
4     Iterator iter = map.entrySet().iterator();  
5     while (iter.hasNext()) {  
6         Map.Entry <Integer, String> entry = (Map.Entry<Integer, String>) iter.next();  
7         cities.add(entry.getValue()); }  
8  
9     assert cities.size() == 5; }
```

We need the cast `(Map.Entry<Integer, String>) iter.next()` to get the next iteration.

2.18.2 Using `for each` over the cities

Using a `for each` loop.

```
1 @Test  
2 public void testMapIteration(){  
3     List <String> cities = new ArrayList <>();  
4     for (Map.Entry <String , Integer> entry : map.entrySet()){  
5         cities.add(entry.getValue()); }
```

```
6
7  assert cities.size() == 6; }
```

Using Java 8s `forEach` Over Map Entries

While the Java 7 release was rather boring for developers, Java 8 has brought us some really nice features to work with. We have a new `forEach` statement we can use in conjunction with lambdas. Here is an example of using Java 8 lambdas to iterate over Map entries.

```
1 @Test
2 public void testMapIteration() {
3     List<String> cities = new ArrayList<>();
4     map.forEach((k, v) -> cities.add(v));
5     assert cities.size() == 6; }
```

You can see Java 8 really allows us to clean up the code. No casts here. You can also see we're skipping over explicitly dealing with the `Map.Entry` object in the lambda. Even though we're taking a shortcut here, we still have the type safety of Java.

Appendix A

Java's `int hashCode` method

Here is some interesting correspondence concerning the `int hashCode()` function in Java's `String` class.

JDK-4045622: `java.lang.String.hashCode` spec incorrectly describes the hash algorithm

Details Type: Bug

Submit Date: 1997-04-17

Status: Closed

Updated Date: 1999-01-15

Project Name: JDK

Resolved Date: 1999-01-15

Component: specification

OS: solaris_2.5

Sub-Component: language

CPU: sparc

Priority: P2

Resolution: Fixed

Affected Versions: 1.1.1

Fixed Versions: 1.2.0 (1.2alpha)

Related Reports

Sub Tasks

Description

The JLS 20.12.10 (page 535) describes the `java.lang.String.hashCode` method in great detail. Unfortunately, it doesn't describe what the code actually

does. The JLS says that for short strings the `hashCode` is (excuse the made up syntax for integer exponentiation):

$$((c[0] * 37^0) + (c[1] * 37^1) + (c[2] * 37^2) + \dots)$$

and something similar, but sampling, for long strings. In fact, the code we have been shipping since 1994 is

```

1 public int hashCode() {
2     int h = 0;
3     int off = offset;
4     char val[] = value;
5     int len = count;
6
7     if (len < 16) {
8         for (int i = len ; i > 0; i--)
9             h = (h * 37) + val[off++]; }
10    else { // only sample some characters
11        int skip = len / 8;
12        for (int i = len ; i > 0; i -= skip, off += skip)
13            h = (h * 39) + val[off]; }
14    return h; }

```

in which the exponentiation has been strength-reduced to multiplication, but in the process is applied to the characters of the string in reverse order.

Here's a test program that shows the difference:

```

1 % cat java/HashTest.java
2 public class HashTest {
3     public static void main(String[] argv) {
4         String one = new String("\u0001");
5         int oneHash = one.hashCode();
6         System.out.println("oneHash:_" + Integer.toString(oneHash));
7         int oneJLS = (((int)'\u0001') * ((int)Math.pow(37.0, 0))));
8         System.out.println("(1*_37^0):_" +
9             Integer.toString(oneJLS));
10
11        String two = new String("\u0001\u0002");
12        int twoHash = two.hashCode();
13        System.out.println("twoHash:_" + Integer.toString(twoHash));
14        int twoJLS = (((int)'\u0001') * ((int)Math.pow(37.0, 0))) +
15            (((int)'\u0002') * ((int)Math.pow(37.0, 1))));
16        System.out.println("(1*_37^0)_(2*_37^1):_" +
17            Integer.toString(twoJLS));
18
19        String three = new String("\u0001\u0002\u0003");
20        int threeHash = three.hashCode();
21        System.out.println("threeHash:_" + Integer.toString(threeHash));

```

```

22 int threeJLS = (((int) '\u0001') * ((int) Math.pow(37.0, 0))) +
23   (((int) '\u0002') * ((int) Math.pow(37.0, 1))) +
24   (((int) '\u0003') * ((int) Math.pow(37.0, 2)));
25 System.out.println("(1_*_37^0)_+_ (2_*_37^1)_+_ (3_*_37^2)):_" +
26   Integer.toString(threeJLS)); } }

```

```

1 % VM12/build/bin/java -classpathclasses:VM12/build/classes HashTest
2 oneHash: 1
3 ((1 * 37^0)): 1
4 twoHash: 39
5 ((1 * 37^0) + (2 * 37^1)): 75
6 threeHash: 1446
7 ((1 * 37^0) + (2 * 37^1) + (3 * 37^2)): 4182

```

You can see that it's off even by the two-character string.

Usually I side with the JLS over the code, but in this case I think the JLS was trying to describe what the code was doing but got it wrong. I don't think we can change the code, since there will be a lot of codes out there that are depending on the unchanging computation of the String.hashCode method (I think of all those serialized hash tables which would be confused if they were deserialized in a VM whose String.hashCode method was different). Either way, something needs to be changed, and quickly.

peter.kessler@Eng 1997-04-16

It turns out that there's another, unrelated bug in the spec: there is an error in the formulae for calculating the increment (k) and maximum value (m) that are used in the sigma expression for the "sampled hash" that is used if $n \geq 15$. As a result, the specified expression references characters that lie out of bounds, and would cause runtime exceptions if correctly implemented.

joshua.bloch@Eng 1997-04-24

Comments CONVERTED DATA

BugTraq+ Release Management Values

COMMIT TO FIX: generic

FIXED IN: 1.2alpha

INTEGRATED IN: 1.2alpha

2004-06-14

SUGGESTED FIX

The following is the accepted proposal for the new string hash function, followed by a report justifying its selection:

New String Hash Function (Bugs 4045622, 1258091)

=====

The Problem: The currently specified String hash function does not match the currently implemented function. The specified function is not implementable. (It addresses characters outside of the input string.) The implemented function performs very poorly on certain classes of strings, including URLs. (The poor performance is due to the "sampling" nature of the function for strings over 15 characters.) I view the specification problem as the perfect opportunity to replace the unfortunate implementation.

Requesters: The problems with the implementation have been mentioned on `comp.sys.java.lang.programmer`, though the extent of the problem may not be known outside of JavaSoft. The problems with the spec were discovered by Peter Kessler and myself.

Proposed API change: No API would change, per se. The function computed by `String.hashCode()` would change to:

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$$

where $s[i]$ is the i th character of string s .

The Java Language Specification (which specifies the value to be returned by `String.hashCode()`) would be modified to reflect this.

The new hash function was selected after a fair amount of study, as described in Exhibit A. In the unlikely event that you want even more detail, see me.

Implementation: Trivial. (4 lines of code, which have already been written.)

Performance impact: Hashing large strings will be somewhat more expensive, as the new hash function examines every character, but Hashtables performance will, on balance, improve, as hash collisions will be vastly reduced. While one could construct a program to demonstrate the reduced speed of the new hash function, I would be very surprised if any real applications were adversely affected in any measurable way.

Risk Assessment: Surprisingly small. Serialization of Hashtables does not depend on consistent hash values. It is possible that some customer has implemented some persistent data structure that relies on the String hash function not changing, but it is highly unlikely that more than a handful of customers have done so.

Testing ImpactP: None, really. I've performed a fair number of tests on the proposed hash function already. (See below.)

Doc Impact: The release notes would have to mention the change, so that any "researchy" customers who do rely on the precise behavior of the hash function would be alerted. The JLS would have to be modified in its next edition. The actual text would be very short (1-2 paragraphs).

Exhibit A

As per Guy's request, here's a note describing the "Pedigree" of my proposed String hash function. Surprisingly, I could find very little in the way of published literature on practical, general-purpose string hash functions; much of what is published is either outdated (e.g., Knuth V. 3, Pp. 508-513) or arcane (lots of journal articles on 'perfect hashing'). Most of the standard algorithms and data structures textbooks barely mention the subject. The best treatment I was able to find was in Aho, Sethi and Ullman's "Compilers: Principles, Techniques and Tools" AKA "The Dragon Book" (Pp. 433-438).

The class of hash function that I recommend using is polynomials whose coefficients are the characters in the string:

$$P(x) = s[0] * x^{(n-1)} + s[1] * x^{(n-2)} + \dots + s[n-1]$$

This polynomial is calculated by the following code fragment:

```
1 int hashVal = 0;
2 for (int i=0; i<string.length; i++)
3     hashVal = x*hashVal + s.charAt(i);
```

The value of **x** is part of the definition of the hash function; choosing this value is somewhat of a black art. Clearly 1 or any multiple of 2 is bad. Primes seem safer than composite numbers, though some composite numbers yield excellent performance (as shown below).

While this class of hash function is recommended in The Dragon Book (P(65599)) and Kernighan and Ritchie's "The C Programming Language, 2 Ed." (P(31)), it is not attributed in either of these books. I went so far as to call up Dennis Ritchie, who said that he did not know where the hash function came from. He walked across the hall and asked Brian Kernighan,

who also had no recollection. A shareware library from Germany called "Matpak" contains ten string hash functions, including P(33), which it attributes to noted Unix hacker Chris Torek. I sincerely doubt that it's original with him. P(37) is used in the definition of the current Java String hash function for short (≤ 16 characters) strings. I'm afraid that the origins of this little function are lost to history.

So why do I think we should use this function? Simply put, it's the best general purpose string hash function that I was able to find, and it's cheap to calculate. By 'general purpose', I mean that it's not optimized for any specific type of strings (e.g., compiler symbols), but seems to resist collisions across every collection of strings that I was able to throw at it. This is critical given that we have no idea what sort of strings people will store in Java hash tables. Also, the performance of this class of hash functions seems largely unaffected by whether the size of the hash table is prime or not. This is important because Java's auto-growing hash table is typically does not contain a prime number of buckets.

In addition to the class of hash functions described above, The Dragon Book highly recommends a slightly more complex hash function due to P. J. Weinberger, originally used internally in his C compiler. Matpak's author, Dr. Berndt M. Gammel (of Technische Universität München), however, claims that Weinberger's function does not work well for large collections of arbitrary strings. It turns out that Matpak's implementation of Weinberger's function contains a typo, and the resulting function is far, far worse even than the currently implemented Java string hash function (see table below). I notified Dr. Gammel, and he promised to fix the typo. The published version of Weinberger's function (in the Dragon book) differs slightly from the version that he (Peter Weinberger) personally recommends. (They differ in the number of bits to which the hash value is shifted: the published value is 24, the recommended value is 28.) The recommended version is significantly better (see Table).

Gammel recommends only three of the ten hash functions in Matpak for "general use." One is P(33), one is a minor variant on P(129), ascribed to Phong Vo, and the third is a table lookup based, S-box like algorithm, relying on a table of 256 "random" 32-bit numbers, from the WAIS project. (The Vo variant merely adds the constant 987654321 to each character to form the polynomial coefficients.)

The table below summarizes the performance of the various hash functions described above, for three data sets:

1) All of the words and phrases with entries in Merriam-Webster's 2nd Int'l Unabridged Dictionary (311,141 strings, avg length 10 chars).

2) All of the strings in /bin/*, /usr/bin/*, /usr/lib/*, /usr/ucb/* and /usr/openwin/bin/* (66,304 strings, avg length 21 characters).

3) A list of URLs gathered by a web-crawler that ran for several hours last night (28,372 strings, avg length 49 characters).

The performance metric shown in the table is the "average chain size" over all elements in the hash table (i.e., the expected value of the number of key compares to look up an element).

	Webster's	Code Strings	URLs
Current Java Fn.	1.2509	1.2738	13.2560
P(37) [Java]	1.2508	1.2481	1.2454
P(65599) [Aho et al]	1.2490	1.2510	1.2450
P(31) [K+R]	1.2500	1.2488	1.2425
P(33)[Torek]	1.2500	1.2500	1.2453
Vo's Fn	1.2487	1.2471	1.2462
WAIS Fn	1.2497	1.2519	1.2452
Weinberger's Fn(MatPak)	6.5169	7.2142	30.6864
Weinberger's Fn(24)	1.3222	1.2791	1.9732
Weinberger's Fn(28)	1.2530	1.2506	1.2439

Looking at this table, it's clear that all of the functions except for the current Java function and the two broken versions of Weinberger's function offer excellent, nearly indistinguishable performance. I strongly conjecture that this performance is essentially the "theoretical ideal", which is what you'd get if you used a true random number generator in place of a hash function.

I'd rule out the WAIS function as its specification contains pages of random numbers, and its performance is no better than any of the far simpler functions. Any of the remaining six functions seem like excellent choices, but we have to pick one. I suppose I'd rule out Vo's variant and Weinberger's function because of their added complexity, albeit minor. Of the remaining four, I'd probably select P(31), as it's the cheapest to calculate on a RISC machine (because 31 is the difference of two powers of two). P(33) is similarly cheap to calculate, but its performance is marginally worse, and 33 is composite, which makes me a bit nervous.

Josh

From: Guy Steele - Sun Microsystems Labs <###@###.###>

To: jbloch@Eng

Cc: gls@East, James.Gosling@Eng, wnj@central, ###@###.###,
riggs@East, ###@###.###

Subject: Re: String Hash Function Redux (long)

Date: Mon, 28 Apr 1997 09:47:22 -0400 (EDT)

Good work! Besides the fact that you have clearly established that we ought to revamp Java's hash function and have found a good solution, I think you have an MPU here (Minimum Publishable Unit)—that is, I would encourage you to write this up even more carefully and send it to an ACM conference (PLDI?) or journal.

—Guy

2004-06-11

PUBLIC COMMENTS

The String hash function implemented in JDK releases through 1.1.3 did not match the function specified in the first edition of the Java Language Specification. In fact, the specified function is not implementable, in that it addresses characters outside of the input string. Additionally, the implemented function performed very poorly on certain classes of strings, including URLs. (The poor performance is due to the "sampling" nature of the function for strings over 15 characters.)

To bring the implementation into accord with the specification, and to fix the performance problems, we are modifying the specification and implementation in tandem. The new String hash function as of JDK1.2 is specified to be:

$$s[0] * 31^{(n-1)} + s[1] * 31^{(n-2)} + \dots + s[n-1]$$

where $s[i]$ is the i th character of string s .

This change should have no effect on the overwhelming majority of Java applications. If an application has persistent data that depends on actual String hash values, it could theoretically be affected, but the serialized representation of a Hashtable does NOT depend on the actual hash values of the keys stored in the Hashtable. Thus, applications relying on serialization for storage of persistent data will be unaffected.

2004-06-10

EVALUATION

The spec and implementation do not match. The specified function CANNOT be implemented, as it would throw `StringIndexOutOfBoundsException`s. Thus, we must change the spec. I feel strongly that it would be in our best interests to replace the hash function with one that does NOT "sample" strings, but includes every character in the hash value computation

joshua.bloch@Eng 1997-04-24

Draft